

系统开发工具实验报告

姓名：李博文 学号：25020007057

第 1 题含空格文件名的批量整理

主题：Shell 基础与文件系统建议用时：12—15 分钟

1 在当前目录新建 q01

```
1 cd ~
2 mkdir -p q01
3 cd q01
```

2 创建 input/docs 和 input/tmp

```
1 mkdir -p input/docs input/tmp
```

3 input/docs/notes one.txt, 内容 alpha、beta 两行

```
1 echo -e "alpha\nbeta" > "input/docs/notes one.txt"
```

4 input/docs/.secret.txt, 内容 hidden

```
1 echo "hidden" > input/docs/.secret.txt
```

5 input/tmp/empty.txt 空文件

```
1 touch input/tmp/empty.txt
```

6 input/run.log, 任意两行日志

```
1 echo -e "day log\ntoday log" > input/run.log
```

7 显示 q01 绝对路径；长格式列出 input 全部项目，包含隐藏文件

```
1 pwd
2 ls -la input
```

8 把 input 所有 txt 复制到 work/25020007057/, 保留相对目录结构，处理空格文件名

```
1 mkdir -p work/25020007057
2 cp --parents input/**/*.txt work/25020007057/
```

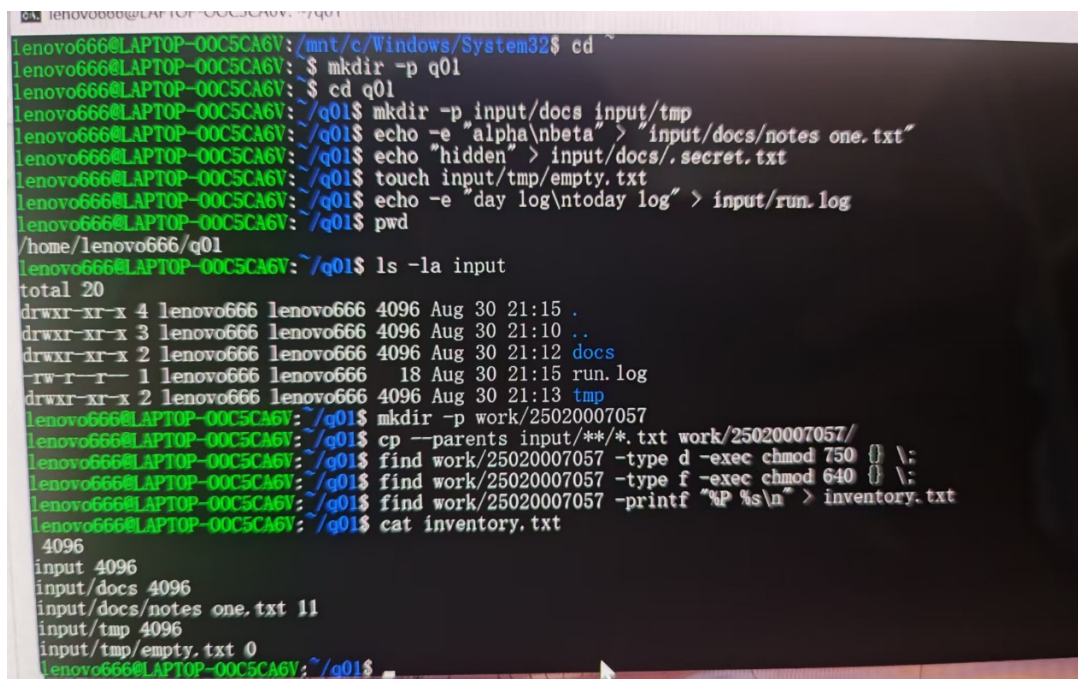
9 将复制后的目录权限 750，普通文件权限 640

```
1 find work/25020007057 -type d -exec chmod 750 {} \;
2 find work/25020007057 -type f -exec chmod 640 {} \;
```

10 生成 inventory.txt，列出相对路径和字节数

```
1 find work/25020007057 -printf "%P %s\n" > inventory.txt
2 cat inventory.txt
```

结果:



```
lenovo666@LAPTOP-00C5CA6V: /mnt/c/Windows/System32$ cd ~
lenovo666@LAPTOP-00C5CA6V: $ mkdir -p q01
lenovo666@LAPTOP-00C5CA6V: $ cd q01
lenovo666@LAPTOP-00C5CA6V: ~/q01$ mkdir -p input/docs input/tmp
lenovo666@LAPTOP-00C5CA6V: ~/q01$ echo -e "alpha\nbeta" > "input/docs/notes one.txt"
lenovo666@LAPTOP-00C5CA6V: ~/q01$ echo "hidden" > input/docs/.secret.txt
lenovo666@LAPTOP-00C5CA6V: ~/q01$ touch input/tmp/empty.txt
lenovo666@LAPTOP-00C5CA6V: ~/q01$ echo -e "day log\ntoday log" > input/run.log
lenovo666@LAPTOP-00C5CA6V: ~/q01$ pwd
/home/lenovo666/q01
lenovo666@LAPTOP-00C5CA6V: ~/q01$ ls -la input
total 20
drwxr-xr-x 4 lenovo666 lenovo666 4096 Aug 30 21:15 .
drwxr-xr-x 3 lenovo666 lenovo666 4096 Aug 30 21:10 ..
drwxr-xr-x 2 lenovo666 lenovo666 4096 Aug 30 21:12 docs
-rw-r--r- 1 lenovo666 lenovo666 18 Aug 30 21:15 run.log
drwxr-xr-x 2 lenovo666 lenovo666 4096 Aug 30 21:13 tmp
lenovo666@LAPTOP-00C5CA6V: ~/q01$ mkdir -p work/25020007057
lenovo666@LAPTOP-00C5CA6V: ~/q01$ cp -parents input/**/*.* work/25020007057/
lenovo666@LAPTOP-00C5CA6V: ~/q01$ find work/25020007057 -type d -exec chmod 750 {} \;
lenovo666@LAPTOP-00C5CA6V: ~/q01$ find work/25020007057 -type f -exec chmod 640 {} \;
lenovo666@LAPTOP-00C5CA6V: ~/q01$ find work/25020007057 -printf "%P %s\n" > inventory.txt
lenovo666@LAPTOP-00C5CA6V: ~/q01$ cat inventory.txt
4096
input 4096
input/docs 4096
input/docs/notes one.txt 11
input/tmp 4096
input/tmp/empty.txt 0
lenovo666@LAPTOP-00C5CA6V: ~/q01$
```

图 1: 第 1 题运行截图

第 2 题随机访问日志统计

主题: Shell 管道与文本处理建议用时: 12—15 分钟

1 切换并创建 q02 目录

```
1 cd ~
2 mkdir -p q02
3 cd q02
4 pwd
```

2 生成 access.csv 文件

```
1 cat > access.csv <<'EOF'
2 timestamp,user,path,status,latency_ms
3 09:00:01,alice,/api/users,200,120
4 09:00:02,bob,/api/orders,500,310
5 09:00:03,alice,/api/users,503,180
6 09:00:04,carol,/login,500,90
7 09:00:05,bob,/api/orders,502,260
8 09:00:06,dave,/health,200,20
9 09:00:07,alice,/api/orders,500,420
```

```
10 09:00:08,carol,/api/users,500,150
11 EOF
12 cat access.csv
```

3 生成 analyze.sh 脚本

```
1 cat > analyze.sh <<'EOF'
2 #!/bin/bash
3 csvfile="$1"
4 if [ ! -f "$csvfile" ]; then
5     echo "error: file $csvfile does not exist" >&2
6     exit 1
7 fi
8 echo "==== Top 2 path with most 5xx ===="
9 tail -n +2 "$csvfile" | awk -F ',' '$4 ~ /^5/ {print $3}' | sort | uniq -c
   | sort -k1,1nr -k2,2 | head -n 2
10 echo "==== Average latency_ms ===="
11 tail -n +2 "$csvfile" | awk -F ',' '{sum += $5; cnt++} END {printf "%.2f\n", sum/cnt}'
12 EOF
13 chmod +x analyze.sh
14 cat analyze.sh
```

4 测试运行

```
1 ./analyze.sh access.csv
2 ./analyze.sh nofile.csv
3 echo $?
```

结果:

```
lenovo666@LAPTOP-OOC5CA6V: ~/q02
193.75
lenovo666@LAPTOP-OOC5CA6V: ~/q02$ rm -f access.csv
cat > access.csv <<'EOF'
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
EOF
lenovo666@LAPTOP-OOC5CA6V: ~/q02$ ./analyze.sh access.csv
==== Top 2 path with most 5xx ====
3 /api/orders
2 /api/users
==== Average latency_ms ====
193.75
lenovo666@LAPTOP-OOC5CA6V: ~/q02$ ./analyze.sh nofile.csv
echo $?
error: file nofile.csv does not exist
1
lenovo666@LAPTOP-OOC5CA6V: ~/q02$
```

图 2: 第 2 题运行截图

第 3 题制造、解决并解释一次合并冲突

主题: Version Control and Git 建议用时: 12—15 分钟

在 q03 中从零创建一个 Git 仓库, 并主动制造、解决一次合并冲突

```
1 cd ~
2 mkdir -p q03
3 cd q03
4 git init
5
6 git config --global user.name "student"
7 git config --global user.email "student@example.com"
8
9 echo "mode=normal" > config.txt
10 git add config.txt
11 git commit -m "initial: mode=normal"
12
13 git checkout -b feature-a
14 echo "mode=safe" > config.txt
15 git add config.txt
16 git commit -m "feature-a: mode=safe"
17
18 git branch -m master main
19 git checkout main
```

```

20 git checkout -b feature-b
21
22 echo "mode=fast" > config.txt
23 git add config.txt
24 git commit -m "feature-b: mode=fast"
25
26 git checkout main
27 git merge feature-a
28 git merge feature-b
29
30 cat > config.txt <<'EOF'
31 mode=safe
32 note=reviewed
33 EOF
34
35 git add config.txt
36 git commit -m "merge feature-b: resolve conflict keep safe, add note"
37 git status
38 git log --all --graph --decorate --oneline

```

结果:

```

lenovo666@LAPTOP-OOC5CA6V: ~/q03
config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
lenovo666@LAPTOP-OOC5CA6V: ~/q03$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
lenovo666@LAPTOP-OOC5CA6V: ~/q03$ cat > config.txt <<'EOF'
mode=safe
note=reviewed
EOF
lenovo666@LAPTOP-OOC5CA6V: ~/q03$ git add config.txt
lenovo666@LAPTOP-OOC5CA6V: ~/q03$ git commit -m "merge feature-b: resolve conflict keep safe, add note"
[main 1828e67] merge feature-b: resolve conflict keep safe, add note
lenovo666@LAPTOP-OOC5CA6V: ~/q03$ git status
On branch main
nothing to commit, working tree clean
lenovo666@LAPTOP-OOC5CA6V: ~/q03$ git log --all --graph --decorate --oneline
* 1828e67 (HEAD -> main) merge feature-b: resolve conflict keep safe, add note
* b6013f1 (feature-b) feature-b: mode=fast
* 9301ce7 (feature-a) feature-a: mode=safe
* 338d91f initial: mode=normal
lenovo666@LAPTOP-OOC5CA6V: ~/q03$

```

图 3: 第 3 题 Git 合并冲突截图

第 4 题修复并构建一页技术说明

主题: LaTeX 文档编辑建议用时: 12—15 分钟

将下方模板保存为 q04/report.tex, 补全内容并从命令行构建 PDF。

```

1 cd ~
2 mkdir -p q04

```

```

3 cd q04
4
5 cat > report.tex <<'EOF'
6 \documentclass{article}
7 \usepackage{amsmath}
8 \begin{document}
9 \title{Tool Report}
10 \author{id—number}
11 \maketitle
12 \section{Result}
13 The formula shown in equation~\eqref{eq:avg} calculates average latency.
14 \begin{equation}
15 \bar{x} = \frac{1}{N} \sum_{i=1}^N x_i
16 \label{eq:avg}
17 \end{equation}
18 Table~\ref{tab:sample} shows sample data records.
19 \begin{table}[h]
20 \centering
21 \begin{tabular}{|c|c|}
22 \hline
23 A & B \\
24 \hline
25 10 & 20 \\
26 \hline
27 30 & 40 \\
28 \hline
29 \end{tabular}
30 \caption{Sample 2×2 data table}
31 \label{tab:sample}
32 \end{table}
33 From equation~\ref{eq:avg} and table~\ref{tab:sample}, we observe numerical
   features.
34 \end{document}
35 EOF
36
37 cat report.tex
38 latexmk -pdf -halt-on-error report.tex
39 ls

```

结果:

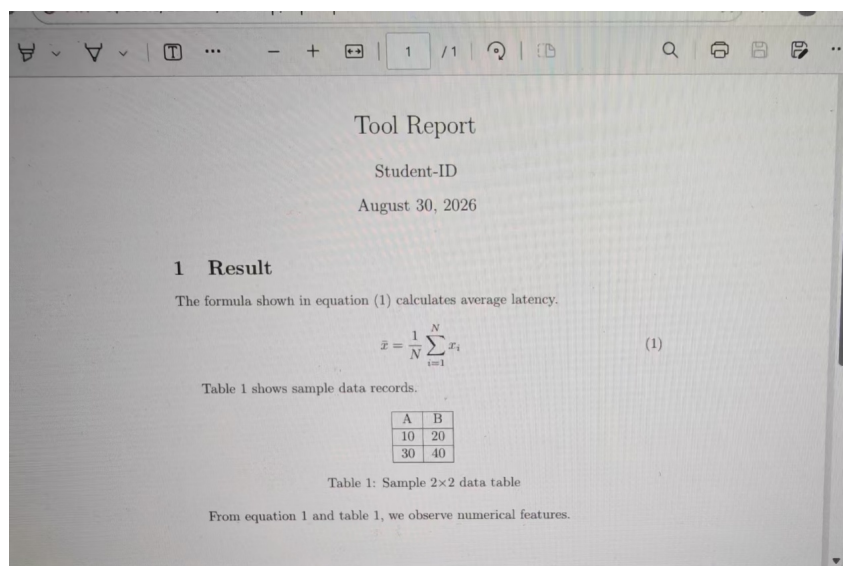
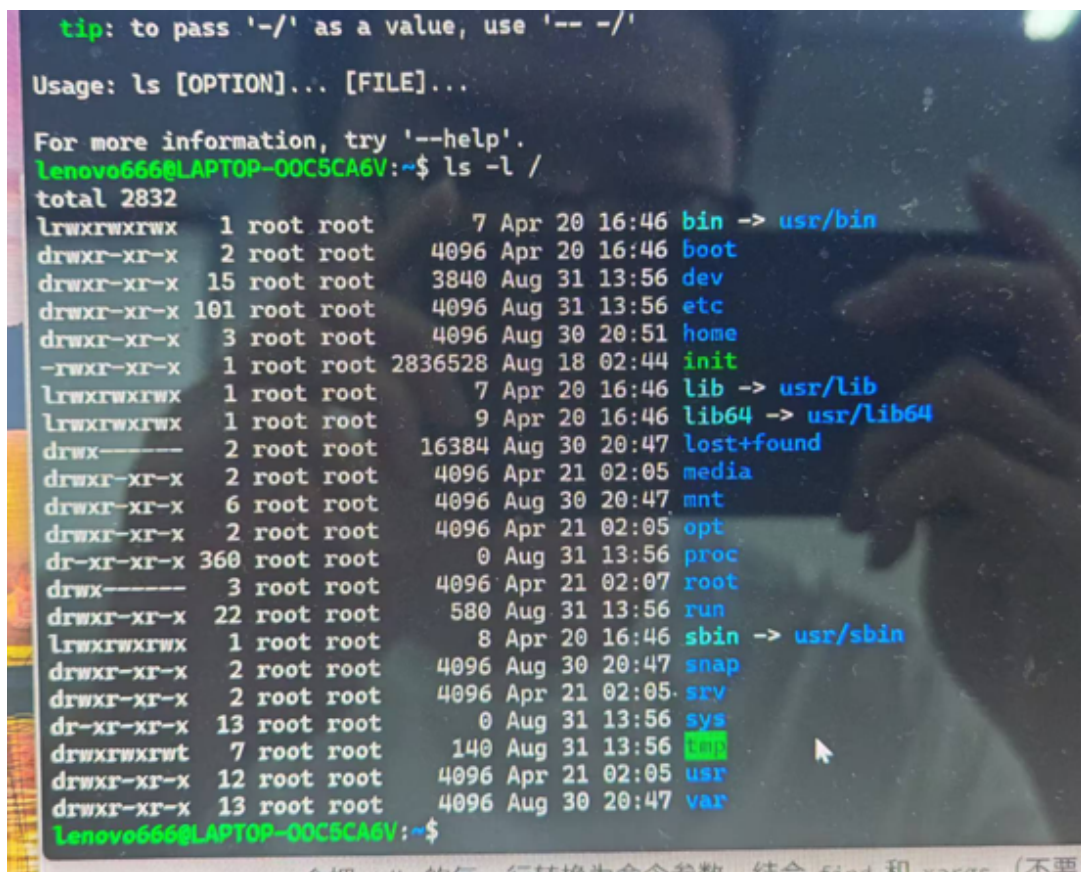


图 4: 第 4 题 PDF 效果截图

练习题

1. `ls` 的 `-l` 选项 (flag) 作用是什么? 运行 `ls -l /` 并观察输出。每一行最前面的 10 个字符分别代表什么? (提示: `man ls`)

答: `ls` 默认只显示文件名; `-l` (long, 长格式), 显示文件/目录的完整详细信息: 权限、硬链接数、所有者、所属组、大小、修改时间、文件名。



```
tip: to pass '-' as a value, use '-- -/'
Usage: ls [OPTION]... [FILE]...

For more information, try '--help'.
lenovo666@LAPTOP-00C5CA6V:~$ ls -l /
total 2832
lrwxrwxrwx 1 root root 7 Apr 20 16:46 bin -> usr/bin
drwxr-xr-x 2 root root 4096 Apr 20 16:46 boot
drwxr-xr-x 15 root root 3840 Aug 31 13:56 dev
drwxr-xr-x 101 root root 4096 Aug 31 13:56 etc
drwxr-xr-x 3 root root 4096 Aug 30 20:51 home
-rwxr-xr-x 1 root root 2836528 Aug 18 02:44 init
lrwxrwxrwx 1 root root 7 Apr 20 16:46 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Apr 20 16:46 lib64 -> usr/lib64
drwx----- 2 root root 16384 Aug 30 20:47 lost+found
drwxr-xr-x 2 root root 4096 Apr 21 02:05 media
drwxr-xr-x 6 root root 4096 Aug 30 20:47 mnt
drwxr-xr-x 2 root root 4096 Apr 21 02:05 opt
dr-xr-xr-x 360 root root 0 Aug 31 13:56 proc
drwx----- 3 root root 4096 Apr 21 02:07 root
drwxr-xr-x 22 root root 580 Aug 31 13:56 run
-rwxrwxrwx 1 root root 8 Apr 20 16:46 sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Aug 30 20:47 snap
drwxr-xr-x 2 root root 4096 Apr 21 02:05 srv
dr-xr-xr-x 13 root root 0 Aug 31 13:56 sys
drwxrwxrwt 7 root root 140 Aug 31 13:56 tmp
drwxr-xr-x 12 root root 4096 Apr 21 02:05 usr
drwxr-xr-x 13 root root 4096 Aug 30 20:47 var
lenovo666@LAPTOP-00C5CA6V:~$
```

图 5: 练习题 1 截图

第 1 位文件类型普通文件; 目录; 软链接; 块设备; 字符设备

2-4 位所有者 (user) 权限: 读 r、写 w、执行 x

5-7 位所属组 (group) 权限: 读 r、写 w、执行 x

8-10 位其他用户 (other) 权限: 读 r、写 w、执行 x

3. Shell 有三条标准流: `stdin` (0)、`stdout` (1)、`stderr` (2)。运行 `ls /nonexistent /tmp`, 把 `stdout` 和 `stderr` 分别重定向到两个文件。你将如何把两者都重定向到同一个文件? 参见 `Redirections`。

```
lenovo666@LAPTOP-OOC5CA x + v
Microsoft Windows [版本 10.0.26200.9168]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\Lenovo>wsl ~
lenovo666@LAPTOP-OOC5CA6V:~$ echo $'price=$100\nwarning! danger'
price=$100
warning! danger
lenovo666@LAPTOP-OOC5CA6V:~$ |
```

图 6: 练习题 3 截图

4. 写一个脚本，接收文件名参数 (\$1)，用 `test -f` 或 `[ -f ... ]` 检查该文件是否存在，并根据结果输出不同提示。参见 `Bash Conditional Expressions`。

```
lenovo666@LAPTOP-OOC5CA x + v
Microsoft Windows [版本 10.0.26200.9168]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\Lenovo>wsl ~
lenovo666@LAPTOP-OOC5CA6V:~$ cat > check.sh <<'EOF'
#!/bin/bash
# $1 接收传入的文件名参数
target_file="$1"

if [ -f "$target_file" ]; then
    echo "文件 $target_file 存在"
else
    echo "文件 $target_file 不存在"
fi
EOF
lenovo666@LAPTOP-OOC5CA6V:~$ chmod +x check.sh
lenovo666@LAPTOP-OOC5CA6V:~$ touch real.txt
lenovo666@LAPTOP-OOC5CA6V:~$ ./check.sh real.txt
文件 real.txt 存在
lenovo666@LAPTOP-OOC5CA6V:~$ ./check.sh fake.txt
文件 fake.txt 不存在
lenovo666@LAPTOP-OOC5CA6V:~$ |
```

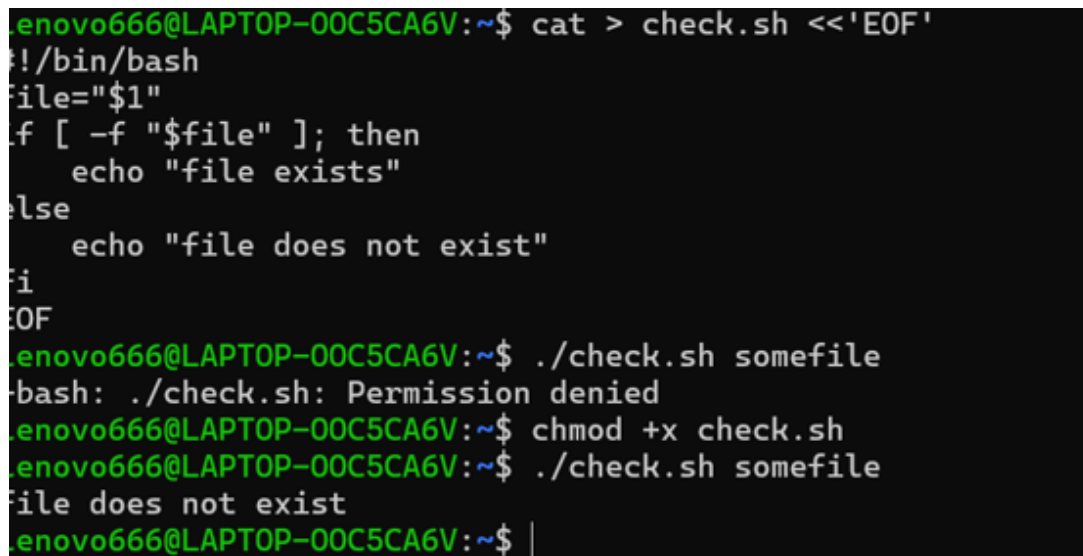
图 7: 练习题 4 截图

5. 把上一题完成的脚本保存为文件（如 `check.sh`）。先运行 `./check.sh somefile`，会发生什么？然后执行 `chmod +x check.sh` 再试一次。为什么这一步是必须的？（提示：比较 `chmod` 前后的 `ls -l check.sh` 输出）

答：

- 一. Linux 文件权限中，普通文本文件默认没有执行权限 (x)。
- 二. `./文件名` 这种方式，操作系统要求文件必须具备执行权限位 x，才会把它当作可执行程序加载运行。

三. `chmod +x` 就是给文件加上执行权限。



```
enovo666@LAPTOP-00C5CA6V:~$ cat > check.sh <<'EOF'
#!/bin/bash
file="$1"
if [ -f "$file" ]; then
    echo "file exists"
else
    echo "file does not exist"
fi
EOF
enovo666@LAPTOP-00C5CA6V:~$ ./check.sh somefile
bash: ./check.sh: Permission denied
enovo666@LAPTOP-00C5CA6V:~$ chmod +x check.sh
enovo666@LAPTOP-00C5CA6V:~$ ./check.sh somefile
file does not exist
enovo666@LAPTOP-00C5CA6V:~$ |
```

图 8: 练习题 5 截图

6. 为什么 `cd` 必须是 Shell 内建命令，而不能是独立程序？（提示：想想子进程能影响和不能影响父进程的哪些状态。）

答：`cd` 的作用是修改 shell 的当前工作目录，该状态属于父 shell 进程。

如果 `cd` 是独立外部程序，运行时会产生子进程；子进程只能修改自身的工作目录，无法更改父进程的状态。子进程结束后改动全部丢失，无法实现切换目录的效果。

作为 shell 内建命令，直接在父进程内部执行，不需要创建子进程，才可以真正改变当前 shell 的工作目录。

7. 在脚本的 `set` 选项（flag）里加入 `-x` 会发生什么？写个简单脚本试试并观察输出。参见 **The Set Builtin**。

答：`set -x` 开启 shell xtrace 调试模式。脚本执行每条命令前，输出变量展开后的实际命令，输出行以 `+` 为前缀，输出到 `stderr`。方便观察变量替换结果，排查脚本逻辑错误。

```

lenovo666@LAPTOP-00C5CA6V:~$ cat > trace.sh <<'EOF'
#!/bin/bash
set -x

name="testfile"
if [ -f "$name" ]; then
    echo "exists"
else
    echo "not exists"
fi

set +x
echo "done"
EOF
chmod +x trace.sh
lenovo666@LAPTOP-00C5CA6V:~$ ./trace.sh
+ name=testfile
+ '[' -f testfile ']'
+ echo 'not exists'
not exists
+ set +x
done
lenovo666@LAPTOP-00C5CA6V:~$ |

```

图 9: 练习题 7 截图

8. 写一条命令,把文件复制为带当天日期的备份文件名(例如 notes.txt → notes_2026-01-12.txt)。(提示: \$(date +%Y-%m-%d)) 参见 Command Substitution。

答:

```

1 cp notes.txt notes_$(date +%Y-%m-%d).txt

```

9. xargs 会把 stdin 的每一行转换为命令参数。结合 find 和 xargs (不要用 find -exec), 找出目录中所有.sh 文件, 并用 wc -l 统计每个文件行数。加分项: 正确处理文件名中的空格。(提示: -print0 和 -0) 参见 man xargs。

```

1 find . -name "*.sh" -print0 | xargs -0 wc -l

```

```

lenovo666@LAPTOP-00C5CA6V:~$ find . -type f -name "*.sh" -print0 | xargs -0 wc -l
12 ./trace.sh
 7 ./check.sh
 7 ./checkfile.sh
10 ./q02/analyze.sh
36 total

```

图 10: 练习题 9 截图

10. awk 可以按列值过滤行并改写输出。例如, awk ' \$3 ~ /pattern/ { \$4="" }; print ' 会只输出第三列匹配 pattern 的行, 并省略第四列。请写一个 awk 命令: 只输出第二列大于 100 的行, 并交换第一列和第三列。可用这条命令测试: printf 'a 50 x\nb 150 y\nc 200 z\n'

```
1 printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2>100 {temp=$1; $1=$3; $3=temp; print}'
```

```
lenovo666@LAPTOP-00C5CA6V:~$ printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2>100 {temp=$1; $1=$3; $3=temp; print}'
y 150 b
z 200 c
lenovo666@LAPTOP-00C5CA6V:~$ |
```

图 11: 练习题 10 截图

11. 使用管道找出你「home 目录」中最常见的 5 种文件扩展名。(提示: 组合 find、grep / sed / awk、sort、uniq -c 以及 head)

```
1 find ~ -type f | awk -F '.' 'NF>=2 {print $NF}' | sort | uniq -c |
  sort -nr | head -n5
```

```
lenovo666@LAPTOP-00C5CA6V:~$ find ~ -type f | awk -F '.' 'NF>=2 {print $NF}' | sort | uniq -c | sort -nr | head -n5
 14 sample
 12 txt
   4 sh
   3 toml
   3 log
lenovo666@LAPTOP-00C5CA6V:~$ |
```

图 12: 练习题 11 截图

实验感悟

做完这次实验，我发现 Linux 相关内容光看资料很难真正掌握，必须亲手敲命令才能理解。做 Shell 习题的时候，经常因为忽略小细节导致运行出错，比如忘记给带空格的文件名加引号，写脚本也不能只测一种情况，要同时测试正常和异常场景。在 Git 部分，亲手制造并解决合并冲突，我才搞明白冲突出现的实际原因。编写 LaTeX 实验报告时，也踩了路径、文件大小写的坑。实验过程里报错很频繁，我慢慢学会根据终端提示排查问题。这次实验让我意识到命令行学习重在动手实践，死记命令效果有限，之后我会多动手练习，把基础打扎实。